

Федеральное государственное автономное образовательное учреждение высшего
образования
«Национальный исследовательский университет ИТМО»

Лабораторная работа №1

по дисциплине:
«Численные методы алгебры»

Тема:
«Исследование влияния предобработки данных на качество линейной
регрессии»

Выполнили:
Студенты группы J3119
Андреасян Егор
Захаров Егор

Санкт-Петербург, 2026

Вариант 1

Параметры: `test_size = 0.30`, `random_state = 41`

1. Цель работы

Предложить среднеквадратическую аппроксимацию табличной функции многих переменных, проанализировать чувствительность точного решения к ошибкам округления, проверить сходимость расчётных и исходных данных.

2. Теоретические сведения

2.1. Линейная регрессия и связь со СЛАУ

В общем самая простая реализация и вид линейной регрессии такой

$$f(x) = Xw$$

X - матрица, наши данные где колонки - это признаки, а строки - просто следующие данные которые надо рассчитать.

w - наши веса.

В теории лабы написано что есть еще случайная ошибка, но в большинстве источников все ее сокращают и убирают из формулы.

Теперь связь со СЛАУ и МНК и тд, в чем вообще приколы, если приглядеться то можно увидеть вообще такую картину

$$Xw = y$$

По факту наше СЛАУ - ищем вектор весов w чтобы получить наши таргеты y .

Теперь причем тут МНК, если мы просто вычислим w то какое-то отличие все равно будет, и поэтому можно применить МНК, вычисляя веса так чтобы ошибка была минимальной.

Ошибка выглядит так

$$error = ||Xw - y||^2$$

распишем

$$\begin{aligned} error &= ||Xw - y||^2 = (Xw - y)^T (Xw - y) = \\ &= (w^T X^T - y^T)(Xw - y) = w^T X^T Xw - w^T X^T y - y^T Xw + y^T y \end{aligned}$$

$$w^T X^T y = (w^T X^T y)^T = y^T Xw$$

$$error = w^T X^T Xw - 2w^T X^T y + y^T y$$

Теперь посчитаем градиент

$$\nabla error_w = 2X^T Xw - 2X^T y + 0$$

$$\nabla error_w = 2X^T Xw - 2X^T y$$

Теперь нам нужен минимум ошибки, что делаем? очев

$$\nabla error_w = 0$$

$$2X^T X w - 2X^T y = 0$$

$$2X^T X w = 2X^T y$$

и тут мы получили СЛАУ, ищем мы w

$$w = (X^T X)^{-1} X^T y$$

и по факту все, это как один из способов, либо можно искать веса через градиентный спуск.

МНК это вообще наш критерий - что он говорит: минимизируем квадрат ошибки и все.

2.2. StandardScaler

StandardScaler - обычное масштабирование данных. Формула такая:

$$x' = \frac{x - \mu}{\sigma}$$

где μ - среднее по признаку, σ - стандартное отклонение. У него два параметра: `with_mean` - вычитать ли среднее, `with_std` - делить ли на стандартное отклонение. Если включить оба, все признаки будут с нулевым средним и единичной дисперсией, и матрица $X^T X$ будет нормально обусловлена.

Зачем это нужно - без масштабирования у нас признаки типа `population` это тысячи, а `latitude` это десятки, и из-за этого собственные значения $X^T X$ сильно отличаются друг от друга, число обусловленности улетает.

2.3. PolynomialFeatures

PolynomialFeatures - это уже прикольная штука, полиномиальные комбинации, т.е. мы берем все наши фичи и делаем разные взаимодействия над каждой фичей либо над фичами в паре и тд, возводим в квадрат или перемножаем друг с другом и тд, зависит от `degree`.

Например если у нас признаки x_1, x_2 и `degree=2`, то получим: $x_1, x_2, x_1^2, x_1 x_2, x_2^2$. При `degree=3` ещё добавятся кубы и тройные комбинации. Это позволяет линейной модели ловить нелинейные зависимости - по сути модель остаётся линейной, но по новым (полиномиальным) признакам.

Проблема в том что с ростом degree количество признаков растёт очень быстро, и они начинают сильно коррелировать друг с другом, из-за чего число обусловленности улетает.

2.4. Метрики качества

MAE (Mean Absolute Error) - средняя абсолютная ошибка, показывает насколько в среднем наши предсказания отличаются от реальных значений:

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

Число обусловленности матрицы - показывает насколько решение СЛАУ чувствительно к ошибкам в данных:

$$\text{cond}(A) = \|A\| \cdot \|A^{-1}\|$$

Если $\text{cond}(X^T X)$ большое (типа 10^{11}) - это значит что мелкие изменения в данных могут сильно поменять веса, и решение ненадёжное. Если маленькое (типа 180) - всё хорошо, решение стабильное.

3. Ход работы

3.1. Реализация линейной регрессии

```
1 class LinearRegression:
2     def __init__(self):
3         self.w = None
4
5     def fit(self, X, y, fit_var: str='inv',
6             iter: int=1024, learning_rate: float=0.01,
7             eps: float=1e-8, delta: float=1e-8):
8         if fit_var == 'inv':
9             self.w = np.linalg.inv(X.T @ X) @ X.T @ y
10        elif fit_var == 'solve':
11            self.w = np.linalg.solve(X.T @ X, X.T @ y)
12        elif fit_var == 'lstsq':
13            self.w, _, _, _ = np.linalg.lstsq(
14                X, y, rcond=None)
15        elif fit_var == 'grad':
16            self._fit_gradient_lin_reg(
17                X, y, iter, learning_rate, eps, delta)
18        else:
19            raise ValueError(
20                f"Unknown fit_var: {fit_var}")
21
22    def _fit_gradient_lin_reg(self, X, y, iter,
23                              learning_rate, eps, delta):
24        n = X.shape[0]
25        self.w = np.zeros(X.shape[1])
26        best_w = self.w.copy()
27        best_error = np.inf
28        for i in range(iter):
29            pred = X @ self.w
30            error = np.mean(np.abs(pred - y) ** 2)
31            grad = (2 / n) * X.T @ (pred - y)
32            self.w -= learning_rate * grad
33            if error < best_error:
34                best_error = error
35                best_w = self.w.copy()
36            if error < eps or \
37                np.linalg.norm(grad) < delta:
38                break
39        self.w = best_w
```

```
40  
41     def predict(self, X):  
42         return X @ self.w
```

3.2. Метрики

```
1 def mean_squared_error(y_true, y_pred):
2     return np.mean((y_true - y_pred) ** 2)
3
4 def mean_absolute_error(y_true, y_pred):
5     return np.mean(np.abs(y_true - y_pred))
```

3.3. Загрузка данных и разбиение

```
1 SEED = 41
2 TEST_SIZE = 0.3
3
4 data = pd.read_csv('data/housing.csv', delimiter=';')
5 data.fillna(data.median().astype(int), inplace=True)
6
7 X = data.drop(
8     columns=["median_house_value"]).values
9 Y = data["median_house_value"].values
10
11 X_train, X_valid, Y_train, Y_valid = \
12     train_test_split(
13         X, Y, test_size=TEST_SIZE,
14         random_state=SEED)
```


3.4. Смотрим данные

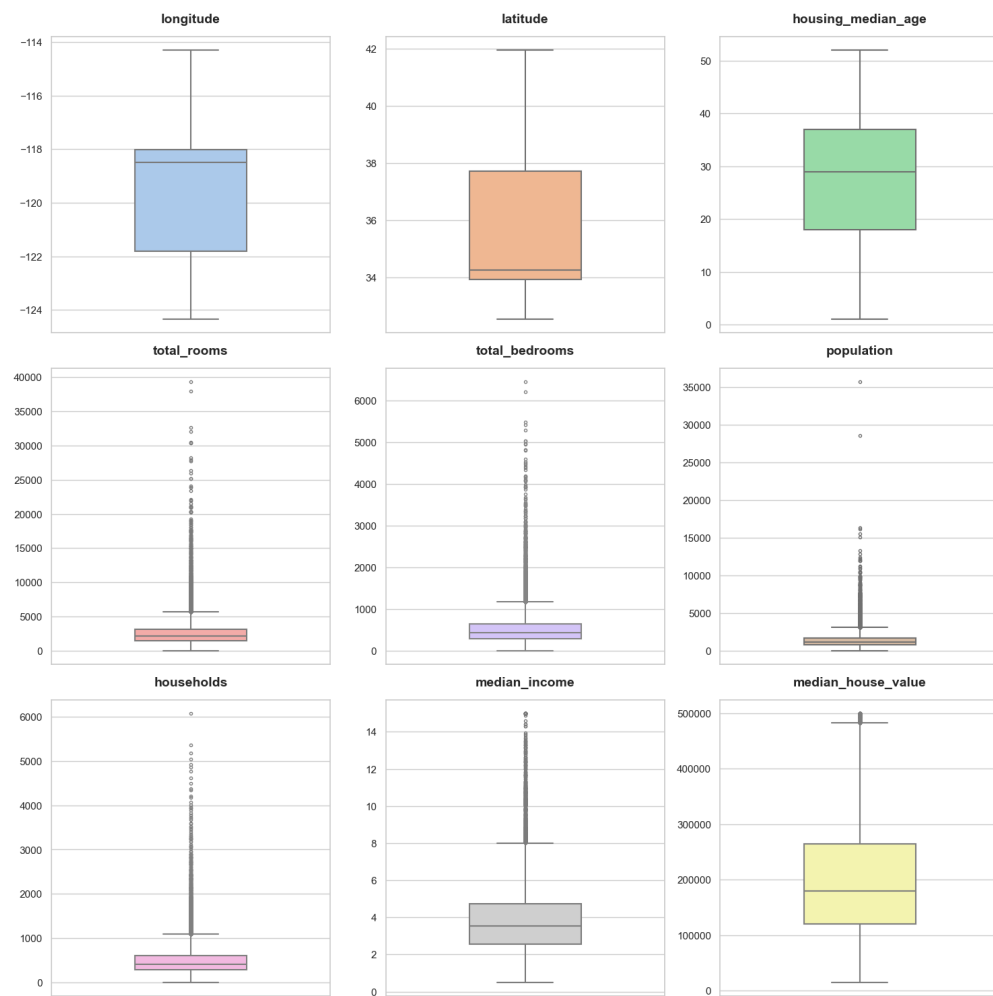


Рис. 1: Вохplot-ы числовых признаков

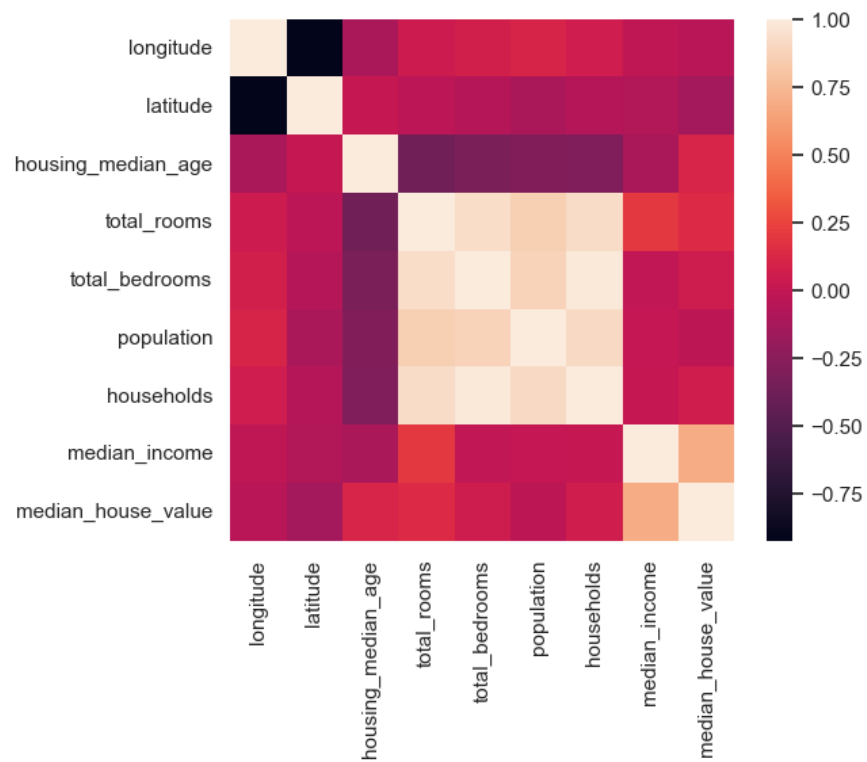


Рис. 2: Корреляционная матрица

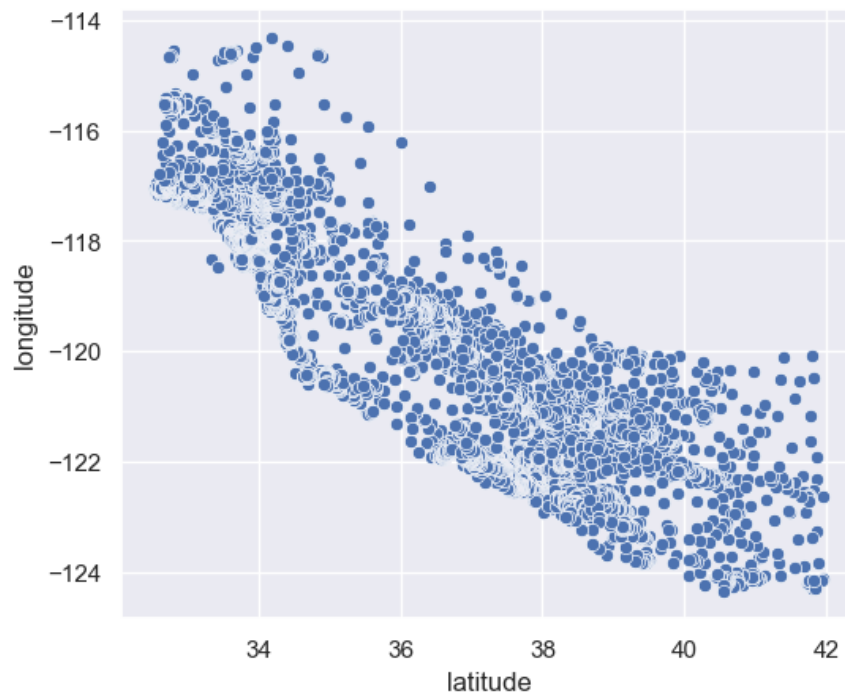


Рис. 3: Распределение координат

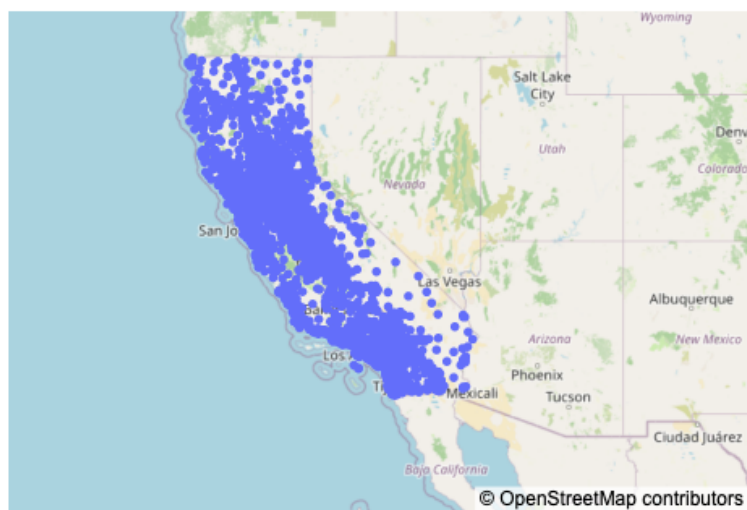


Рис. 4: Карта данных, Калифорния

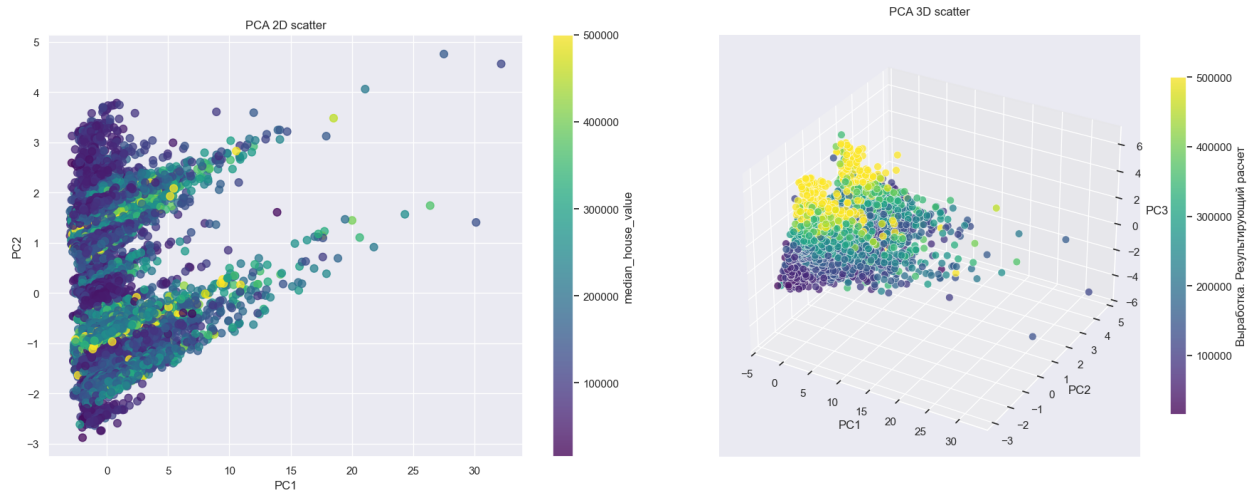


Рис. 5: PCA: 2D и 3D scatter

3.5. Эксперименты с предобработкой

Пример эксперимента (mean=False, std=False, degree=1):

```
1 polym_f = PolynomialFeatures(  
2     degree=1, include_bias=False)  
3 X_train_poly = polym_f.fit_transform(X_train)  
4 X_valid_poly = polym_f.transform(X_valid)  
5  
6 scaler = StandardScaler(  
7     with_mean=False, with_std=False)  
8 X_train_scaled = scaler.fit_transform(X_train_poly)  
9 X_valid_scaled = scaler.transform(X_valid_poly)  
10  
11 X_train_scaled = np.hstack(  
12     [np.ones((X_train_scaled.shape[0], 1)),  
13     X_train_scaled])  
14 X_valid_scaled = np.hstack(  
15     [np.ones((X_valid_scaled.shape[0], 1)),  
16     X_valid_scaled])
```

Аналогично для остальных комбинаций.

3.6. Таблица результатов

Таблица 1: Результаты экспериментов

№	mean	std	deg	MAE train	MAE test	$k(X^T X)$
1	F	F	1	50980.23	51213.53	$2.6 \cdot 10^{11}$
2	T	F	1	50980.23	51213.53	$2.4 \cdot 10^7$
3	F	T	1	50980.23	51213.53	$6.4 \cdot 10^7$
4	T	T	1	50980.23	51213.53	$1.8 \cdot 10^2$
5	T	T	2	45061.84	45512.36	$5.3 \cdot 10^7$
6	T	T	3	41621.87	43085.55	$6.3 \cdot 10^{12}$

4. Анализ результатов

4.1. Как масштабирование влияет на качество?

MAE в экспериментах 1–4 одинаковый (train: 50980, test: 51213). Аналитическое решение $w^* = (X^T X)^{-1} X^T y$ находит одну и ту же модель - масштабирование меняет масштаб весов, но предсказания остаются теми же.

Однако масштабирование критически влияет на число обусловленности:

- Без стандартизации: $k \approx 2.6 \cdot 10^{11}$
- Только mean=True: $k \approx 2.4 \cdot 10^7$
- Только std=True: $k \approx 6.4 \cdot 10^7$
- Полная (mean+std): $k \approx 180$

При полной стандартизации все признаки приводятся к одному масштабу, и матрица становится хорошо обусловленной.

4.2. Качество при увеличении degree

- degree=1: MAE_test = 51213
- degree=2: MAE_test = 45512
- degree=3: MAE_test = 43085

PolynomialFeatures добавляет нелинейные комбинации признаков (x_i^2 , $x_i x_j$, x_i^3 и т.д.), что позволяет модели лучше аппроксимировать реальную зависимость.

4.3. Признаки переобучения

Да, переобучение проявляется при увеличении degree:

- degree=1: разница MAE train/test = 233
- degree=2: разница = 451
- degree=3: разница = 1464

Разница растёт - модель запоминает обучающие данные, но хуже обобщает на новые.

4.4. Число обусловленности и качество

При полной стандартизации (mean+std):

- degree=1: $k \approx 180$
- degree=2: $k \approx 5.3 \cdot 10^7$
- degree=3: $k \approx 6.3 \cdot 10^{12}$

С ростом degree число обусловленности растёт экспоненциально, потому что полиномиальные признаки высоких степеней сильно коррелированы (x_i^2 и x_i^3 почти линейно зависимы), матрица $X^T X$ становится почти вырожденной.

4.5. Лучшая комбинация

По MAE лучший результат в эксперименте №6 (degree=3) с MAE_test = 43085. Но $k \approx 6.3 \cdot 10^{12}$ - решение численно нестабильно. Оптимальный компромисс - эксп. №5 (degree=2) с MAE_test = 45512 и $k \approx 5.3 \cdot 10^7$.

4.6. Графики

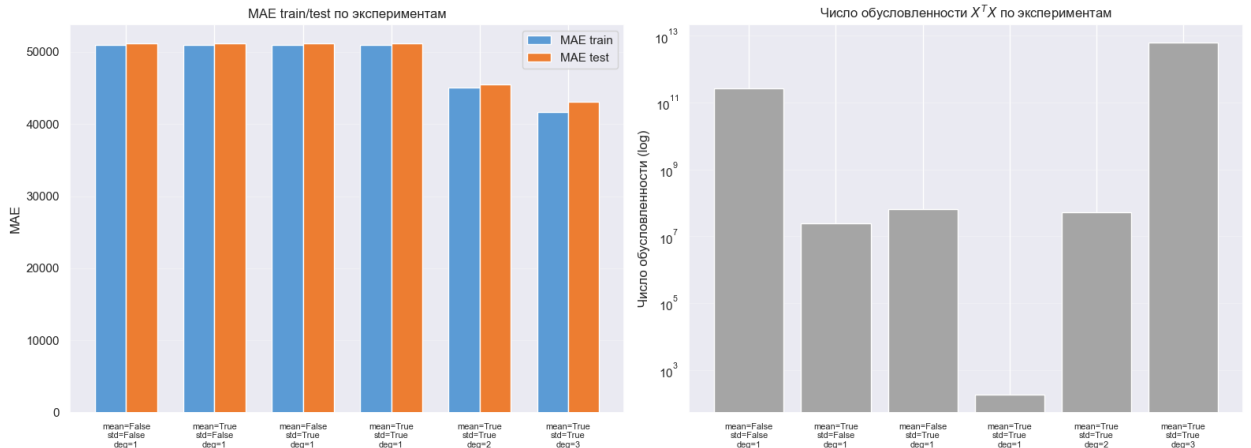


Рис. 6: MAE train/test и число обусловленности

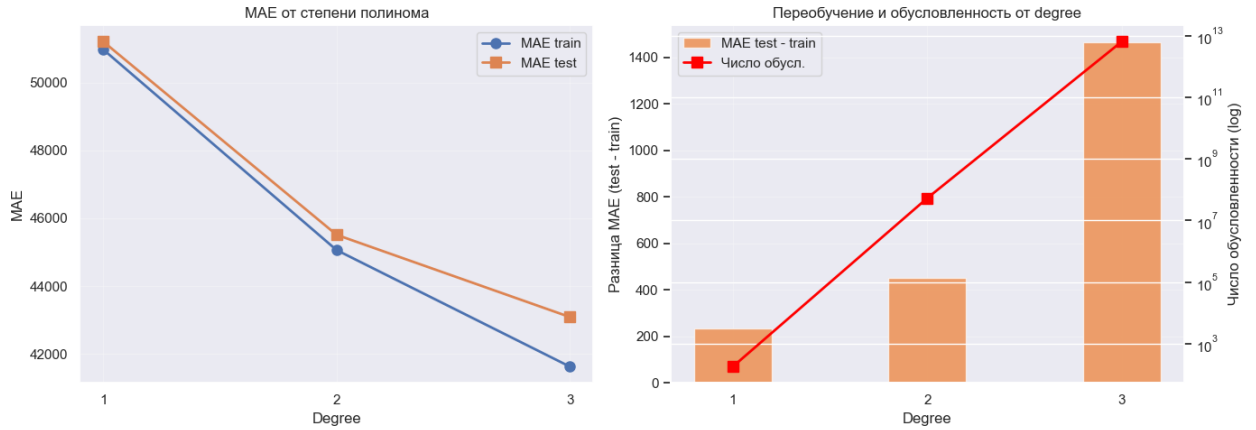


Рис. 7: MAE от степени полинома

5. Выводы

В ходе лабораторной работы была реализована линейная регрессия с нуля, выведена формула оптимальных весов через метод наименьших квадратов и проведены эксперименты с различными параметрами предобработки данных.

Было показано, что задача линейной регрессии сводится к решению системы нормальных уравнений $X^T X w = X^T y$, которая получается из условия минимума функции потерь $\|Xw - y\|^2$. Реализованы три способа решения этой СЛАУ: через обратную матрицу (inv), прямое решение системы (solve) и SVD-разложение (lstsq) - все три дают одинаковый результат на хорошо обусловленных данных.

Анализ числа обусловленности матрицы $X^T X$ подтвердил, что без стандартизации признаков матрица близка к вырожденной ($k \approx 10^{11}$), что делает решение чувствительным к ошибкам округления. Полная стандартизация (mean + std) снижает число обусловленности до $k \approx 180$, обеспечивая численную устойчивость.

Эксперименты с полиномиальными признаками показали, что увеличение степени полинома улучшает качество аппроксимации (MAE_test снижается на 16% при degree=3), но одновременно ведёт к росту числа обусловленности и разрыву между ошибками на обучающей и тестовой выборках - признакам переобучения. Оптимальной комбинацией параметров является полная стандартизация с degree=2, обеспечивающая баланс между качеством модели и устойчивостью решения.